Functional Programming Assignment

Parsing, Validating, and Generating Quiz Papers from XML in Haskell

Module - Functional Programming
Assignment Title - Quiz Bank Parser and Paper Generator

Table of Contents

Overview

In this assignment, you will design and implement a Haskell program that reads one or more Moodle-style XML quiz files, converts them into an internal question bank, validates important invariants, and generates useful outputs from that question bank.

The assignment brings together several core ideas from the module, including algebraic data types, parsing, list processing, validation, randomisation, and property-based testing.

The overall aim is to move from **structured external data** to a **well-designed internal model**, and then from that model to **validated generated outputs**.

---

1. Learning Goals

By completing this assignment, you should be able to:

- model structured data using suitable algebraic data types
- parse XML-like hierarchical data into Haskell values
- design and enforce invariants over imported and generated data
- transform structured data into Markdown output
- generate randomised outputs subject to constraints
- use QuickCheck to test important program properties
- separate parsing, validation, transformation, and generation into clear program components

---

2. Assignment Context

You will be given:

- introductory material on XML structure
- examples of parsing techniques in Haskell
- starter code (including some sample quizzes in xml form)

All supporting slide decks, some recorded are available on the Functional Programming tutors site here.

To access the starter code, please use this link here.

Each XML file will contain a default quiz category and a set of questions. Some questions may also specify their own explicit category.

Your program must load these files and store the questions internally.

A key design rule is that **every stored question must have a category**. If a question has no explicit category, it must inherit the default category from its quiz file.

---

3. Core Task

You are asked to download (clone) starter code from GitHub Classroom and develop and build a Haskell program using that structure that does the following:

Part A: Import and model the question bank

Your program must read one or more XML files and parse them into suitable Haskell data types.
At minimum, each stored question should contain:

- question type
- category
- question name or title where present
- question text
- answers
- feedback

The category rule is as follows:

- if a question has an explicit category, use it
- otherwise, inherit the default quiz category

Every stored question must therefore end up with exactly one category.

Part B: Generate a Markdown file containing all questions

Your program must generate a Markdown file containing:

- all questions
- numbering for each question
- the question text
- the category of each question

Part C: Generate a Markdown file containing all questions, answers, and feedback

Your program must generate a second Markdown file containing:

- numbered questions
- category for each question
- answer options
- indication of correctness where appropriate
- feedback where present

Part D: Generate shuffled question sets

Your program must generate randomly shuffled sets of questions.

It should be possible to generate shuffled outputs of different sizes, for example:

- 5 questions
- 10 questions
- all questions in random order

Generated question sets must still be clearly numbered.

---

4. Extension: Valid Paper Generation

After building the basic importer and Markdown generator, you must extend the system so that it can generate a **valid paper** from the question bank subject to lecturer-defined constraints.

This is a required part of the assignment.

At minimum, you must extend your internal model to include:

- **marks per question**

You may also add other useful attributes if you wish, such as difficulty, estimated time, or tags, but marks are the required extension.

---

5. Invariants

A major part of this assignment is the use of **invariants**.
You must identify, document, and enforce important rules over both the question bank and the generated paper.

5.1 Question-bank invariants

Examples include:

- every stored question has a category
- every stored question has question text
- marks are positive
- a multiple-choice question has at least one answer
- a multiple-choice question has at least one correct answer

You may define additional sensible invariants where appropriate.

5.2 Paper-generation invariants

Your program must support lecturer-defined constraints including:

- a minimum number of questions from each category
- a required total mark value for the paper

For example, a lecturer might require:

- at least 2 questions from Introduction
- at least 3 questions from Loops
- at least 2 questions from Arrays
- total marks must equal 20

Your generated paper must satisfy all such specified constraints.

The selected questions must be shuffled before output.

5.3 Failure behaviour

If the constraints cannot be satisfied, your program must fail clearly and report the problem appropriately.

It must not silently generate an invalid paper.

---

6. QuickCheck Requirement

You must use **QuickCheck** to test important properties of your solution.

QuickCheck is not a substitute for enforcing invariants in your design, but it should be used to test that your core functions preserve the rules you define.

Examples of suitable properties include:

- a missing question category inherits the quiz default
- an explicit question category overrides the quiz default
- shuffling preserves the number of questions
- shuffling preserves the set of questions
- if a paper is generated successfully, it satisfies the required category minima
- if a paper is generated successfully, it has the correct total marks

You should include a suitable set of properties and any generators needed to support them.

7. Program Design Expectations

You are free to design your own solution, but strong solutions will clearly separate:

- parsing
- internal modelling
- validation
- Markdown rendering
- paper generation
- property testing

A sensible module structure might look like:

- `Adts.hs`
- `Parse.hs`
- `Validate.hs`
- `RenderMarkdown.hs`
- `GeneratePaper.hs`
- `Main.hs`

You do not have to use these exact names.

---

8. AI Usage Reflection

You are asked to write a reflection document as described below

8.1 Purpose

This reflection is not about catching you using AI. It is about demonstrating that you engaged with it critically. AI tools can produce plausible-looking Haskell that compiles but is wrong, ignores your invariants, uses the wrong types, or misses the point of the task entirely. Knowing how to recognise that, and fix it, is part of what this module is assessing.

Using AI is permitted. Submitting AI output that you do not understand is not.

8.2 Format and Length

Submit your reflection as "reflection.md"(using markdown - this is the preferred option) or reflection.pdf. It should be **300–500 words**, not counting code excerpts.
Your name and student number should be clearly in the start of the document.

Part 1 — Declaration (approx. 50 words)

State clearly whether you used any AI tools during this assignment. If yes, name them (for example: ChatGPT, Claude, GitHub Copilot, Gemini). If you did not use any AI tools, state this and skip Parts 2 and 3.

Part 2 — Worked Example (approx. 200 words + code)

Choose one specific interaction where AI assisted you on this assignment. It does not have to be the most impressive — it should be the most honest. Include all three of the following.

1. **The prompt you gave:**
   Copy your prompt exactly as you typed it.

2. **What the AI produced:**
   Include a relevant excerpt of the response or code. You do not need to paste everything — just enough to show what it gave you.

3. **What you changed, and why:**
   Explain specifically what you modified, added, or rejected in the AI output. This is the most important part. A vague statement such as “I tidied it up” is not sufficient. Show the before and after if the change was to code.

Examples of the kind of change being looked for:

- The AI ignored the category inheritance rule from Section 2, so you rewrote that part
- The AI used String where your design uses a proper ADT, so you changed the types throughout
- The AI generated a QuickCheck property that always passed trivially, so you replaced it with one that tests the invariant
- The AI produced a valid-paper generator that did not fail clearly when constraints were unsatisfiable, so you added the error handling from Section 5.3

Part 3 — Critical Reflection (approx. 150–200 words)

Reflect briefly on your experience of using AI for this assignment. Address at least two of the following.

- Where was AI most useful? (for example: explaining a concept, producing boilerplate, suggesting a structure)
- Where did it fall short or mislead you? (for example: wrong types, ignoring invariants, not understanding the Moodle XML format)
- Did AI help you move faster, or did fixing its output cost more time than writing from scratch?
- What would you do differently if you were starting again?

Part 4 — References

List each AI tool you used, the approximate date, and a one-line description of what you used it for. Example:

Tool	Approx. Date	Used for
Claude	*March 2026*	*Explaining cursor-based XML navigation*
ChatGPT	*March 2026*	*Draft of the QuickCheck Arbitrary instance*

8.3 Marking

The reflection contributes to the Code Quality and Communication criterion (Section 10.5 of the spec, 15% of the total mark). It will be assessed on:

- Honesty and specificity — a reflection that shows real engagement is worth more than a polished one that says nothing
- Evidence that you understood any AI-generated code you submitted
- Clarity of the worked example

A reflection that is vague, generic, or inconsistent with the code you submitted will attract a low mark in this criterion. A reflection that honestly documents where AI helped and where you had to correct it will attract a high one.

8.4 Important Notes

- You will not be penalised for having used AI, only for not reflecting on it honestly.
- If your reflection suggests you do not understand the code in your submission, that will affect your mark across all criteria, not just this one.
- There is no requirement to have used AI. A genuine “I did not use it” reflection is perfectly valid.

9. Deliverables

You must submit:

- all Haskell source files
- a short README explaining how to build and run the program
- sample XML input file(s)
- a generated Markdown file containing questions only
- a generated Markdown file containing questions, answers, and feedback
- at least one generated shuffled paper
- QuickCheck properties and any supporting generators
- a short design note explaining:
    - your main data types
    - the invariants you chose
    - where and how those invariants are enforced
- A reflection document

---

10. Suggested Development Milestones

Milestone 1: Data modelling and parsing.

I will ask you to submit this as a formative assignment. No marks will go for this submission but it is an opportunity for you to get feedback on your work so far.

You should:

- inspect the XML structure
- define suitable Haskell data types
- parse one XML file into an internal structure
- demonstrate that questions can be extracted successfully

Suggested evidence

- draft ADTs
- parser code
- simple .md file which contains a copy of the data

Milestone 2: Category inheritance and normalisation

You should:

- implement the default-category rule
- ensure every stored question has exactly one category
- combine multiple files if required

Suggested evidence

- examples showing inherited categories
- examples showing question-level category override

Milestone 3: Markdown generation

You should:

- generate a questions-only Markdown file
- generate a full Markdown file with answers and feedback
- number all questions clearly

Suggested evidence

- both Markdown outputs

Milestone 4: Randomisation and subset generation

You should:

- generate shuffled outputs
- support different requested sizes
- ensure numbering remains correct after shuffling

Suggested evidence

- example shuffled outputs of different sizes

Milestone 5: Invariants and extended model

You should:

- extend questions to include marks
- define question-bank invariants
- define paper-generation invariants
- validate imported and generated data

Suggested evidence

- updated data model
- validation functions
- brief invariant list

Milestone 6: Valid paper generation

You should:

- accept a paper specification
- support minimum counts per category
- support required total marks
- fail clearly when no valid paper can be generated

Suggested evidence

- example paper specifications
- successful generated paper
- example failure case

Milestone 7: QuickCheck

You should:

- define suitable generators or `Arbitrary` instances
- write QuickCheck properties for core behaviours
- demonstrate that important invariants are tested

Suggested evidence

- property definitions
- sample test runs

---

11. Assessment Rubric.

This is not finalised but the final rubric won't be far away.

11.1	11.2	11.3	11.4	11.5	11.6	11.7	11.8
Functional Correctness (including correctness of output)	**Data modelling and functional design**	**Invariants and Validation**	**QuickCheck and testing**	**Code Quality including structure**	**Documentation including reflection (and AI documentation)**	**Use of Github classroom**	**Markdown output**
30	14	10	12	15	9	5	5

11.1 Functional correctness — 30%

Assesses whether the program works as specified.

High-quality work will:

- correctly parse the provided XML structure
- correctly assign categories using inheritance and override rules
- generate both Markdown outputs correctly
- generate shuffled question sets correctly
- generate valid papers satisfying lecturer constraints
- fail appropriately when constraints are unsatisfiable
- have working, documented extra functionality

Lower-quality work may:

- only partially parse the data
- mishandle categories
- produce incomplete output
- ignore or inconsistently enforce constraints

11.2 Data modelling and functional design — 14%

Assesses the quality of the Haskell design.

High-quality work will:

- use well-chosen algebraic data types
- model the domain clearly
- separate parsing, validation, and rendering cleanly
- use `Maybe`, `Either`, and other types appropriately
- have working, documented extra modelling

Lower-quality work may:

- rely too heavily on raw strings
- mix unrelated responsibilities together
- have weak or unclear data modelling

11.3 Invariants and validation — 10%

Assesses how well the rules are defined and enforced over the question bank and generated papers.

High-quality work will:

- state invariants clearly
- enforce them in appropriate parts of the code
- validate both imported data and generated outputs
- handle invalid or impossible cases sensibly

Lower-quality work may:

- mention invariants without enforcing them
- enforce them inconsistently
- produce invalid papers without reporting failure

11.4 QuickCheck and testing — 12%

Assesses the use of property-based testing.

High-quality work will:

- include meaningful QuickCheck properties
- test important behaviours and invariants
- use sensible generators
- demonstrate understanding of what QuickCheck can and cannot establish

Lower-quality work may:

- include only trivial properties
- test very little of the important functionality
- rely only on manual examples

11.5 Code quality including structure — 15%

Assesses readability, structure, and explanation and includes assessment of the **reflection document.**

High-quality work will:

- include Haskell program style
- be clearly structured and readable (including good stack structure and corresponding properties.yaml)
- include useful naming and organisation
- compile cleanly
- include a README with clear run instructions
- include a concise design note explaining choices

Lower-quality work may:

- be difficult to follow
- have unclear structure
- lack explanation of design choices or invariants

11.6 Documentation including AI reflection— 9%

Assesses readability, structure, and explanation and includes assessment of the **reflection document.**

High-quality work will:

- be clearly structured and readable
- include a concise design note explaining choices
- include full declaration of AI Bot use etc.

Lower-quality work may:

- be difficult to follow
- lack explanation of design choices or invariants
- lack full disclosure of AI use

11.7 Use of Github Classroom - 5%

High-quality work will:

- Have regular commits
- Commit messages will be well structured and clearly shows the flow of work

Lower-quality work may:

- Have irregular/or few commits
- Commit comments are not well-written - either too long or too short.

For information on well-written commit messages see here

11.8 Markdown output - 5%

In addition to having correct information,

High-quality work will:

- Reports(quizzes, tests) that look well and are easy to read.
- Extra formatting to improve usability and readability

Lower-quality work may:

- Little extra formatting from basics

12. Constraints and Assumptions

For this assignment, you may assume:

- the provided XML files use the Moodle-style structure discussed in class (and given in starter code)
- you only need to support the subset of question structures represented in the provided files, i.e. multiple choice questions
- you are not required to support every Moodle question type
- stronger solutions may handle malformed input more robustly, but full general XML support is not required

---

13. Submission Guidance

Your submission should:

- be submitted by pushing to Github Classroom and also Moodle
- compile successfully
- include all required files (source and yaml) using the stack package.
- include a reflection document which will
    - include a brief design note
    - make clear where invariants are enforced
    - make clear where QuickCheck is used
    -

- include a README with build and run instructions
- include sample outputs (in .md form)

---

14. Summary

You are building a Haskell system that:

- imports Moodle-style XML quiz data
- stores that data in appropriate Haskell types
- ensures each question has a category
- generates numbered Markdown outputs
- generates shuffled question sets
- adds marks per question
- generates valid papers subject to constraints
- uses QuickCheck to test important properties

This is both a parsing assignment and a modelling-and-validation assignment. Strong solutions will treat it as both.
